

Day 12 - 31th July 2026

Task 01:

Why is there a need for DBMS?

- Reduces data redundancy.
- Improves data consistency.
- Ensures data security.
- Maintains data integrity.
- Enables data sharing.
- Supports multiple users.
- Provides backup and recovery.
- Allows efficient data retrieval.
- Simplifies data management.
- Reduces application development time.

Task 02:

Why not traditional DBMS?

- A traditional file system is not suitable because it stores duplicate data, leading to data redundancy and inconsistency. It provides poor security, makes data sharing and retrieval difficult, lacks data integrity and concurrency control, and does not offer proper backup and recovery facilities. As a result, managing and maintaining data becomes difficult and inefficient.

Task 03:

Explain the concept of **data independence** in DBMS

- Data Independence is the ability of a DBMS to modify the database structure without affecting the application programs. It allows changes at one level of the database without requiring changes at higher levels, making the database more flexible and easier to maintain.
- There are two types of data independence: Physical Data Independence, which allows changes in physical storage without affecting the logical structure, and Logical Data Independence, which allows changes in the logical structure without affecting user applications.

Task 04:

What are the different types of **database models** (hierarchical, network, relational, object-oriented)?

Explain with examples

Types of Database Models

1. Hierarchical Database Model:

- Data is organized in a tree-like structure with parent-child relationships.
- Each child record has only one parent record.
- Example: IBM IMS (Information Management System).
- Used in: Organization charts, file systems.

2. Network Database Model:

- Data is organized as a network of records with multiple relationships.
- A child record can have multiple parent records.
- Example: Integrated Data Store (IDS).
- Used in: Complex data relationships.

3. Relational Database Model:

- Data is stored in the form of tables (rows and columns).
- Relationships between tables are created using keys.
- Example: MySQL, Oracle, PostgreSQL.
- Used in: Banking, business applications, websites.

4. Object-Oriented Database Model:

- Data is stored as objects, similar to object-oriented programming.
- Supports features like classes, inheritance, and methods.
- Example: ObjectDB, db4o.
- Used in: Multimedia, engineering, and scientific applications.

Task 05

Describe **concurrency control** in DBMS. Why is it important?

Concurrency Control is a technique in DBMS that manages the simultaneous execution of multiple transactions to ensure data consistency and accuracy.

Importance:

- Prevents data inconsistency.
- Maintains data integrity.
- Avoids problems like lost updates and incorrect data.
- Ensures transactions are executed safely.
- Allows multiple users to access the database at the same time.

Task 06:

What are **indexes** in DBMS? How do they improve query performance?

Indexes are data structures in DBMS that help in faster searching and retrieval of data from a table without scanning the entire table.

How they improve query performance:

- Reduce the time required to search data.
- Allow faster data retrieval.
- Improve the performance of SELECT queries.
- Reduce disk access operations.
- Help in efficient sorting and filtering of data.

Task 07:

Explain **deadlock** in DBMS. What are the strategies for deadlock prevention and detection?

Deadlock is a situation in DBMS where two or more transactions are waiting for each other to release resources, causing all of them to stop executing.

Deadlock Prevention Strategies:

- Avoid holding resources while waiting for others.
- Allocate resources in a fixed order.
- Use timestamp-based methods like Wait-Die and Wound-Wait.
- Lock resources before starting a transaction.

Deadlock Detection Strategies:

- Use a Wait-For Graph to identify cycles.
 - Periodically check for deadlocks.
 - Abort one or more transactions to resolve the deadlock.
 - Release resources held by the selected transaction.
-

RDBMS

Task 08:

What do you understand by Relational model?

Relational Model is a database model that stores data in the form of tables (relations) consisting of rows and columns. Each table represents an entity, rows represent records, and columns represent attributes. Relationships between tables are created using keys such as primary keys and foreign keys.

Example: A Student table with columns like Student_ID, Name, and Course_ID represents student data in a relational database.

Task 09:

Specify key terms that you encounter in RDBMS. Give brief explanation

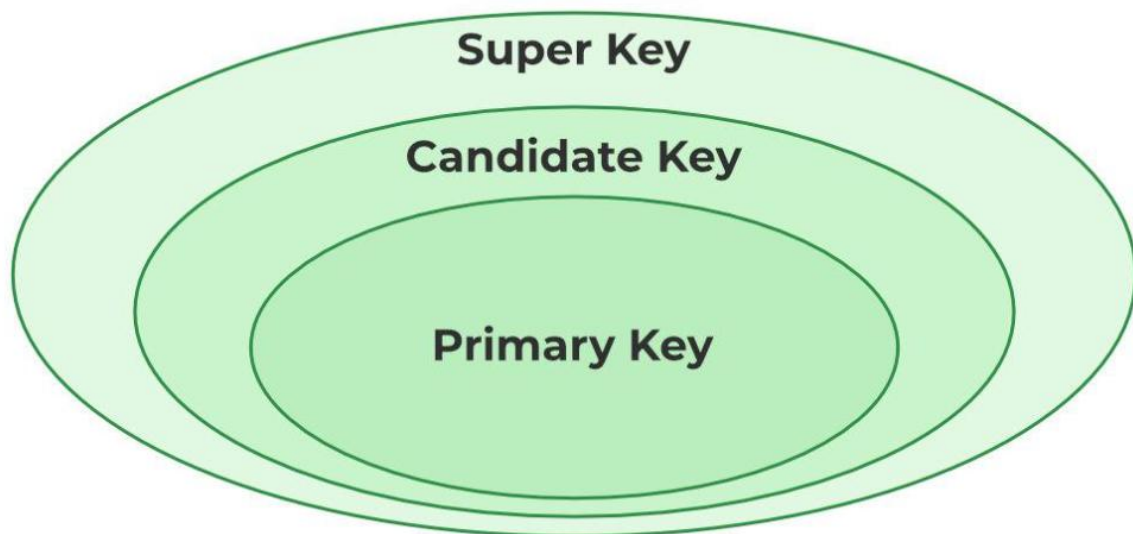
- **Relation:** A table in a database that stores data in rows and columns.
- **Tuple:** A single row or record in a table.
- **Attribute:** A column in a table that represents a property of an entity.
- **Primary Key:** An attribute that uniquely identifies each record in a table.

- **Foreign Key:** An attribute that creates a link between two tables by referring to the primary key of another table.
- **Candidate Key:** An attribute or set of attributes that can uniquely identify a record.
- **Super Key:** A set of attributes that uniquely identifies a record, possibly with extra attributes.
- **Degree:** The number of attributes (columns) in a relation.
- **Cardinality:** The number of tuples (rows) in a relation.
- **Constraint:** Rules applied to maintain data accuracy and integrity.
- **Normalization:** The process of organizing data to reduce redundancy and improve consistency.

Task 10:

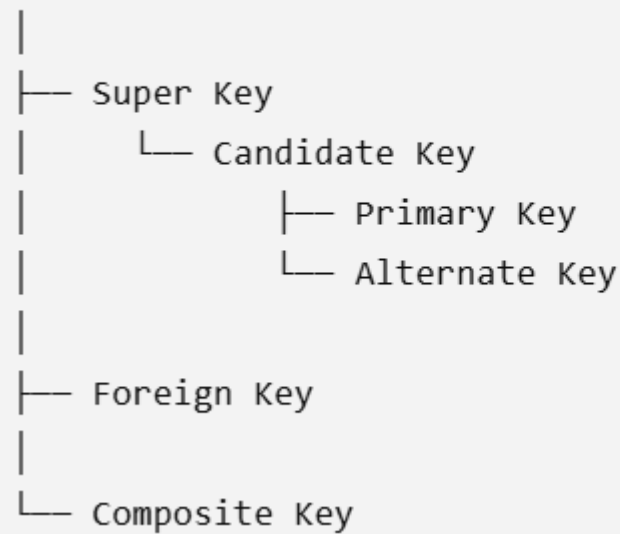
What are different types of keys in RDBMS?

Explain using diagrams ..



Relation between Primary Key, Candidate Key, and Super Key

Keys in RDBMS



Types of keys

- ☐ **Candidate Key:** A key that can uniquely identify each record in a table.
- ☐ **Super Key:** A set of attributes that uniquely identifies a record, with or without extra attributes.
- ☐ **Composite Key:** A key formed using two or more attributes to uniquely identify a record.
- ☐ **Primary Key:** A selected candidate key used to uniquely identify records in a table.
- ☐ **Foreign Key:** A key that creates a relationship between two tables by referencing a primary key.
- ☐ **Alternate Key:** A candidate key that is not chosen as the primary key.

Task 11:

As you know diff keys in RDBMS what is the difference between **primary key**, **foreign key**, and **unique key**?

- A **Primary Key** is a key that uniquely identifies each record in a table and cannot contain duplicate or NULL values.
- A **Foreign Key** is a key that is used to create a relationship between two tables by referring to the primary key of another table and can contain duplicate values. A
- **Unique Key** ensures that all values in a column are unique and prevents duplicate entries, but it may allow NULL values.

Task 12

How does **transaction management** work in DBMS? Give an example of **commit** and **rollback**

- Transaction Management in DBMS is the process of managing database transactions to ensure data consistency, accuracy, and reliability. It controls the execution of multiple operations by using transaction commands like COMMIT and ROLLBACK.

Example:

COMMIT:

When a user transfers ₹1000 from Account A to Account B, after successfully completing the transaction, COMMIT permanently saves the changes in the database.

ROLLBACK:

If an error occurs during the transfer process, ROLLBACK cancels all changes and restores the database to its previous state.

Task 13

What is **normalization**? Why is it necessary in database design?

Normalization is the process of organizing data in a database to reduce data redundancy and improve data consistency by dividing large tables into smaller related tables.

It is necessary because it:

- Reduces duplicate data.
- Improves data consistency.
- Maintains data integrity.
- Avoids insertion, update, and deletion anomalies.
- Makes database design more efficient and easier to maintain.

Task 14

What do you understand by 1nf? Explain with an example

- 1NF states that every column in a table should contain only one (atomic) value, and there should be no repeating groups or multiple values in a single cell.
- Example

Before 1NF (Not in 1NF)

Student_ID	Student_Name	Courses
101	Ravi	DBMS, Java
102	Anu	C, Python

Here, the **Courses** column contains multiple values, so the table is **not in 1NF**.

After 1NF

Student_ID	Student_Name	Course
101	Ravi	DBMS
101	Ravi	Java
102	Anu	C
102	Anu	Python

Task 15

What do you understand about 2NF? Explain with an example.

- 2NF states that a table must be in 1NF, and every non-key attribute must depend on the entire primary key, not just part of it. It removes partial dependency.
- Example

Primary Key = (Student_ID, Course_ID)

Student_ID	Course_ID	Student_Name	Course_Name
101	C01	Ravi	DBMS
101	C02	Ravi	Java
102	C01	Anu	DBMS

Here:

- **Student_Name** depends only on **Student_ID**.
- **Course_Name** depends only on **Course_ID**.

These are **partial dependencies**, so the table is **not in 2NF**.

Student Table	
Student_ID	Student_Name
101	Ravi
102	Anu
Course Table	
Course_ID	Course_Name
C01	DBMS
C02	Java
Enrollment Table	
Student_ID	Course_ID
101	C01
101	C02
102	C01

Now, all non-key attributes depend on the **whole primary key**, so the tables are in **2NF**.

Task 16

What do you understand by 3 nf ? Explain with an example

- 3NF states that a table must be in 2NF, and all non-key attributes must depend only on the primary key and not on another non-key attribute. It removes transitive dependency.

Before 3NF (Not in 3NF)			
Employee_ID	Employee_Name	Department_ID	Department_Name
101	Ravi	D01	CSE
102	Anu	D02	ECE

Here:

- Employee_ID → Department_ID
- Department_ID → Department_Name

Since Department_Name depends on Department_ID (a non-key attribute) instead of directly on Employee_ID, the table is **not in 3NF**.

After 3NF

Employee Table

Employee_ID	Employee_Name	Department_ID
101	Ravi	D01
102	Anu	D02

Department Table

Department_ID	Department_Name
D01	CSE
D02	ECE

Now, all non-key attributes depend **only on the primary key**, so the tables are in **3NF**.

Task 17

What do you understand by joins? Explain all joins with examples

- A **Join** is an SQL operation used to **combine data from two or more tables** based on a common column.

Example Tables

Student Table

Student_ID	Name	Dept_ID
101	Ravi	D01
102	Anu	D02
103	Kiran	D03

Department Table

Dept_ID	Dept_Name
D01	CSE
D02	ECE
D04	EEE

1. Inner Join

- Returns **only the matching records** from both tables.

Result

Student_ID	Name	Dept_Name
101	Ravi	CSE
102	Anu	ECE

2. Left Join (Left Outer Join)

- Returns **all records from the left table** and matching records from the right table.

Result

Student_ID	Name	Dept_Name
101	Ravi	CSE
102	Anu	ECE
103	Kiran	NULL

3. Right Join (Right Outer Join)

- Returns **all records from the right table** and matching records from the left table.

Result

Student_ID	Name	Dept_Name
101	Ravi	CSE
102	Anu	ECE
NULL	NULL	EEE

4. Full Join (Full Outer Join)

- Returns **all matching and non-matching records** from both tables.

Result

Student_ID	Name	Dept_Name
101	Ravi	CSE
102	Anu	ECE
103	Kiran	NULL
NULL	NULL	EEE

5. Self Join

- A table is **joined with itself**.

Example

Employee Table

Emp_ID	Emp_Name	Manager_ID
101	Ravi	NULL
102	Anu	101
103	Kiran	101

Result

Employee	Manager
Anu	Ravi
Kiran	Ravi

6. Cross Join

- Returns the **Cartesian product** of two tables (every row from one table is combined with every row from the other table).

Table A

A

1

2

Table B

B

X

Y

Result

A

B

1

X

1

Y

2

X

2

Y

Home Task

What is BCNF explain with example

- BCNF an advanced form of 3NF. A table is in BCNF if every determinant is a candidate key. In other words, every attribute that determines another attribute must itself be a candidate key.
- Example

Example

Consider the table:

Student	Course	Teacher
Ravi	DBMS	Mr. A
Anu	DBMS	Mr. A
Ravi	Java	Mr. B

Convert to BCNF

Teacher Table

Teacher	Course
Mr. A	DBMS
Mr. B	Java

Student Teacher Table

Student	Teacher
Ravi	Mr. A
Anu	Mr. A
Ravi	Mr. B

Now, every determinant (**Teacher**) is a **candidate key** in its table, so the tables are in **BCNF**.